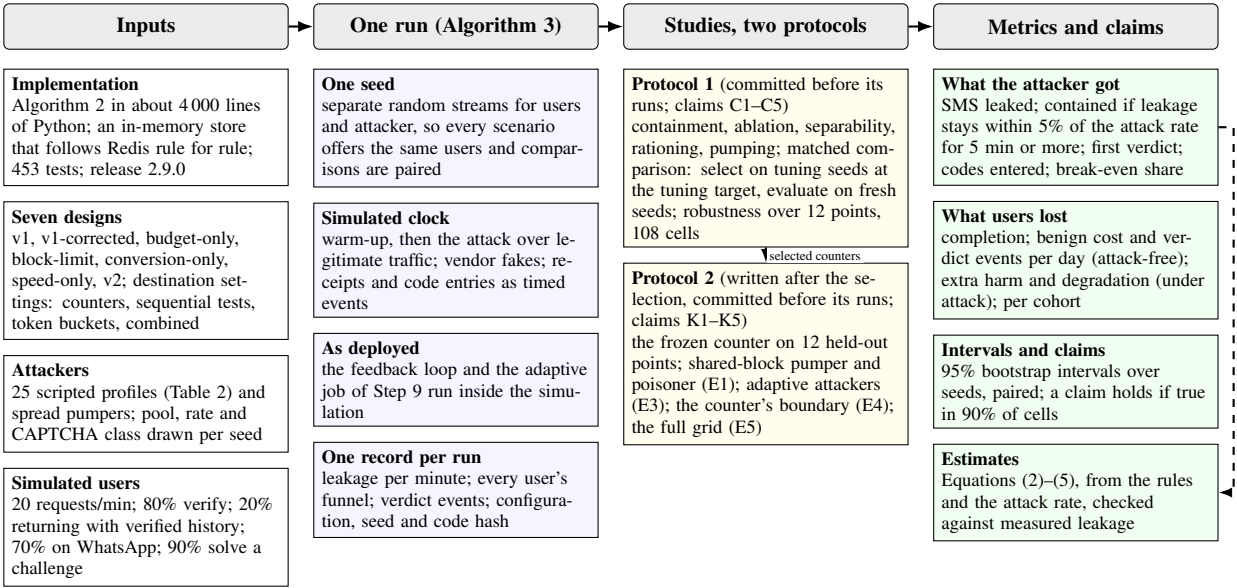
Inputs
One run (Algorithm 3)
Studies, two protocols
Metrics and claims
Implementation
Algorithm 2 in about 4 000 lines of Python; an in-memory store that follows Redis rule for rule; 453 tests; release 2.9.0
Seven designs
v1, v1-corrected, budget-only, block-limit, conversion-only, speed-only, v2; destination settings: counters, sequential tests, token buckets, combined
Attackers
25 scripted profiles (Table 2) and spread pumpers; pool, rate and CAPTCHA class drawn per seed
Simulated users
20 requests/min; 80% verify; 20% returning with verified history; 70% on WhatsApp; 90% solve a challenge
One seed
separate random streams for users and attacker, so every scenario offers the same users and comparisons are paired
Simulated clock
warm-up, then the attack over legitimate traffic; vendor fakes; receipts and code entries as timed events
As deployed
the feedback loop and the adaptive job of Step 9 run inside the simulation
One record per run
leakage per minute; every user's funnel; verdict events; configuration, seed and code hash
Protocol 1 (committed before its runs; claims C1–C5)
containment, ablation, separability, rationing, pumping; matched comparison: select on tuning seeds at the tuning target, evaluate on fresh seeds; robustness over 12 points, 108 cells
selected counters
Protocol 2 (written after the selection, committed before its runs; claims K1–K5)
the frozen counter on 12 held-out points; shared-block pumper and poisoner (E1); adaptive attackers (E3); the counter's boundary (E4); the full grid (E5)
What the attacker got
SMS leaked; contained if leakage stays within 5% of the attack rate for 5 min or more; first verdict; codes entered; break-even share
What users lost
completion; benign cost and verdict events per day (attack-free); extra harm and degradation (under attack); per cohort
Intervals and claims
95% bootstrap intervals over seeds, paired; a claim holds if true in 90% of cells
Estimates
Equations (2)–(5), from the rules and the attack rate, checked against measured leakage